

This notebook fine-tunes and compares three transformer models for the task of suicide ideation detection from Reddit posts.

```
!pip install -q kaggle
from google.colab import files
files.upload()
!mkdir -p ~/.kaggle
!cp kaggle.json ~/.kaggle/
!chmod 600 ~/.kaggle/kaggle.json
!kaggle datasets download -d nikhileswarkomati/suicide-watch
!unzip suicide-watch.zip
import pandas as pd
df = pd.read_csv('Suicide_Detection.csv')
print(f"Dataset shape: {df.shape}")
print(df.head())
```

 Choose Files

No file chosen

Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

Saving kaggle.json to kaggle.json

Dataset URL: <https://www.kaggle.com/datasets/nikhileswarkomati/suicide-watch>

License(s): CC-BY-SA-4.0

Archive: suicide-watch.zip

inflating: Suicide_Detection.csv

Dataset shape: (232074, 3)

Unnamed: 0		text	class
0	2	Ex Wife Threatening SuicideRecently I left my ...	suicide
1	3	Am I weird I don't get affected by compliments...	non-suicide
2	4	Finally 2020 is almost over... So I can never ...	non-suicide
3	8	i need helpjust help me im crying so hard	suicide

```
import pandas as pd
import torch
from torch.utils.data import Dataset, DataLoader
from transformers import AutoTokenizer, AutoModelForSequenceClassification, get_linear_schedule_with_warmup
from torch.optim import AdamW
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report
from tqdm.auto import tqdm

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

class RedditDataset(Dataset):
    def __init__(self, encodings, labels):
        self.encodings = encodings
        self.labels = labels
    def __getitem__(self, idx):
        item = {k: torch.tensor(v[idx]) for k, v in self.encodings.items()}
        item['labels'] = torch.tensor(self.labels[idx])
        return item
    def __len__(self):
        return len(self.labels)

def encode_data(model_name, train_texts, val_texts):
    train_enc = tokenizer(train_texts, truncation=True, padding=True, max_length=256)
    val_enc = tokenizer(val_texts, truncation=True, padding=True, max_length=256)
    return tokenizer, train_enc, val_enc

df = pd.read_csv("Suicide_Detection.csv").dropna()
df['label'] = df['class'].map({'non-suicide': 0, 'suicide': 1})
train_texts, val_texts, train_labels, val_labels = train_test_split(
    df['text'].tolist(), df['label'].tolist(), test_size=0.2, stratify=df['label'], random_state=42
)

def train_and_evaluate(model_name):
    tokenizer, train_enc, val_enc = encode_data(model_name, train_texts, val_texts)
    train_ds = RedditDataset(train_enc, train_labels)
    val_ds = RedditDataset(val_enc, val_labels)
    train_loader = DataLoader(train_ds, batch_size=16, shuffle=True)
    val_loader = DataLoader(val_ds, batch_size=16)
    model = AutoModelForSequenceClassification.from_pretrained(model_name, num_labels=2).to(device)
    optimizer = AdamW(model.parameters(), lr=2e-5)
    total_steps = len(train_loader) * 3
    scheduler = get_linear_schedule_with_warmup(optimizer, num_warmup_steps=0, num_training_steps=total_steps)
    best_val_loss = float('inf')
    for epoch in range(3):
        model.train()
        loop = tqdm(train_loader, desc=f"{model_name} Epoch {epoch+1}/3")
        for batch in loop:
            optimizer.zero_grad()
            batch = {k: v.to(device) for k, v in batch.items()}
            outputs = model(**batch)
            loss = outputs.loss
            loss.backward()
            torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
            optimizer.step()
            scheduler.step()
            loop.set_postfix(loss=loss.item())
        model.eval()
        preds, true, total_val_loss = [], [], 0
        for batch in val_loader:
            batch = {k: v.to(device) for k, v in batch.items()}
            with torch.no_grad():
                outputs = model(**batch)
            total_val_loss += outputs.loss.item()
            logits = outputs.logits
            preds.extend(torch.argmax(logits, dim=1).cpu().numpy())
            true.extend(batch['labels'].cpu().numpy())
        avg_val_loss = total_val_loss / len(val_loader)
        if avg_val_loss < best_val_loss:
            best_val_loss = avg_val_loss
            model.save_pretrained(f"best_{model_name.replace('/', '_')}")
            tokenizer.save_pretrained(f"best_{model_name.replace('/', '_')}")
        report = classification_report(true, preds, target_names=['Non-suicide', 'Suicide'], output_dict=True)
    return report

bert_report = train_and_evaluate("bert-base-uncased")
roberta_report = train_and_evaluate("roberta-base")
```



tokenizer_config.json: 100%

48.0/48.0 [00:00<00:00, 5.91kB/s]

config.json: 100%

570/570 [00:00<00:00, 66.8kB/s]

vocab.txt: 100%

232k/232k [00:00<00:00, 6.53MB/s]

tokenizer.json: 100%

466k/466k [00:00<00:00, 22.2MB/s]

Xet Storage is enabled for this repo, but the 'hf_xet' package is not installed. Falling back to regular HTTP download. For better performance, install the package with: `pip insta

WARNING:huggingface_hub.file_download:Xet Storage is enabled for this repo, but the 'hf_xet' package is not installed. Falling back to regular HTTP download. For better performance

model.safetensors: 100%

440M/440M [00:01<00:00, 240MB/s]

Some weights of BertForSequenceClassification were not initialized from the model checkpoint at bert-base-uncased and are newly initialized: ['classifier.bias', 'classifier.weight']

You should probably TRAIN this model on a down-stream task to be able to use it for predictions and inference.

bert-base-uncased Epoch 1/3: 100%

11604/11604 [30:25<00:00, 6.75it/s, loss=0.000613]

bert-base-uncased Epoch 2/3: 100%

11604/11604 [30:18<00:00, 6.84it/s, loss=0.000304]

bert-base-uncased Epoch 3/3: 100%

11604/11604 [30:18<00:00, 6.87it/s, loss=0.000213]

tokenizer_config.json: 100%

25.0/25.0 [00:00<00:00, 3.30kB/s]

config.json: 100%

481/481 [00:00<00:00, 65.4kB/s]

vocab.json: 100%

899k/899k [00:00<00:00, 13.1MB/s]

merges.txt: 100%

456k/456k [00:00<00:00, 49.4MB/s]

tokenizer.json: 100%

1.36M/1.36M [00:00<00:00, 34.6MB/s]

Xet Storage is enabled for this repo, but the 'hf_xet' package is not installed. Falling back to regular HTTP download. For better performance, install the package with: `pip insta

WARNING:huggingface_hub.file_download:Xet Storage is enabled for this repo, but the 'hf_xet' package is not installed. Falling back to regular HTTP download. For better performance

model.safetensors: 100%

499M/499M [00:02<00:00, 238MB/s]

Some weights of RobertaForSequenceClassification were not initialized from the model checkpoint at roberta-base and are newly initialized: ['classifier.dense.bias', 'classifier.den

You should probably TRAIN this model on a down-stream task to be able to use it for predictions and inference.

roberta-base Epoch 1/3: 100%

11604/11604 [30:33<00:00, 6.60it/s, loss=0.00102]

roberta-base Epoch 2/3: 100%

11604/11604 [30:34<00:00, 6.82it/s, loss=0.000134]

roberta-base Epoch 3/3: 100%

11604/11604 [30:32<00:00, 6.82it/s, loss=3.42e-5]

config.json: 100%

558/558 [00:00<00:00, 69.6kB/s]

vocab.txt: 100%

843k/843k [00:00<00:00, 12.7MB/s]

bpe.codes: 100%

1.08M/1.08M [00:00<00:00, 26.7MB/s]

tokenizer.json: 100%

2.91M/2.91M [00:00<00:00, 36.6MB/s]

emoji is not installed, thus not converting emoticons or emojis into text. Install emoji: pip3 install emoji==0.6.0

pytorch_model.bin: 100%

543M/543M [00:02<00:00, 241MB/s]

Some weights of RobertaForSequenceClassification were not initialized from the model checkpoint at vinai/bertweet-base and are newly initialized: ['classifier.dense.bias', 'classif

You should probably TRAIN this model on a down-stream task to be able to use it for predictions and inference.

model.safetensors: 100%

543M/543M [00:03<00:00, 229MB/s]

vinai/bertweet-base Epoch 1/3: 0%

0/11604 [00:00<?, ?it/s]

```
comparison_df = pd.DataFrame({
    "bert-base": bert_report["Suicide"],
    "roberta-base": roberta_report["Suicide"]#,
}).T[["precision", "recall", "f1-score"]]
display(comparison_df)
```



	precision	recall	f1-score
bert-base	0.979337	0.980308	0.979822
roberta-base	0.996452	0.992330	0.994387